

Sound Zero-Knowledge Proofs for Logarithmic and Periodic Activations in a Lookup-Based Framework

Zhi Chen Xiang

Abstract

Lookup tables are the key tool behind the most efficient VOLE-based zero-knowledge (ZK) proofs for non-linear functions in machine learning [5]. A natural way to extend such a framework is to reuse its pattern for the exponential: split the input into digits, look up each digit, and recombine the results. An earlier master’s thesis [15] sketched a logarithm and a sine in this way, which would enable the softplus, log-softmax and snake activations, but did not implement them. Implementing them now shows that the template needs care. The digit-wise recombination does not compute the logarithm and needs undefined table entries on typical inputs, and the sine’s range reduction must be bound to the input for the proof to be sound. We isolate these two pitfalls and give revised protocols that use only the framework’s building blocks: a logarithm normalised with Msnzb , and a protocol for periodic functions that performs a *verified* reduction modulo one. We prove error bounds of 1–1.5 units in the last place ($\Delta = 2^{-12}$). We implement both protocols in Hao et al.’s released C++ code base (emp-zk). The implementation matches an executable specification bit for bit and rejects every tampered witness. For 10^5 instances, the lookup-based protocols are $4.0\text{--}7.1\times$ faster and use $4.1\text{--}7.1\times$ less communication than the same functions proven without lookups in the same backend.

1 Introduction

ZK proofs let a model owner prove that an inference was computed correctly without revealing the weights. In practice, the non-linear layers are the bottleneck. Hao et al. [5] remove most of this cost with *table lookups* over a VOLE/IT-MAC backend. Inputs are split into 12-bit digits with a verified digit decomposition, and each digit indexes a small public table. Their framework covers ReLU, sigmoid, GELU, softmax and normalisation, and reports $50\text{--}179\times$ speed-ups over Mystique [14].

The author’s master’s thesis [15] (“v1” below) proposed extending the framework with a logarithm $\Pi_{\log}$ and a sine $\Pi_{\sin}$. These are needed for softplus, log-softmax and the periodic snake activation [17], which is used in neural audio codecs [7]. Both protocols follow the template of the exponential $\Pi_{\exp}$: a digit decomposition, one lookup per digit, and a cheap recombination. The thesis presented them as paper designs, without an implementation to check them against.

This paper. We implement both protocols and check them in code. This shows that the template does not carry over directly, and we revise the designs accordingly.

1. **Two pitfalls** (Section 3). (i) Digit-wise lookups only compose through a homomorphism: $e^{-\sum c_i} = \prod e^{-c_i}$, but $\log \sum c_i \neq \sum \log c_i$. The v1 logarithm outputs $\log \prod_i c_i$, and it needs the undefined entry $\log 0$ for every input below 2^{36} . (ii) A committed intermediate that no constraint ties to the input makes a protocol unsound. v1’s sine does not yet tie the reduced angle x_{mod} to the input, so a malicious prover can have any table output accepted.
2. **Sound constructions** (Section 4). Π_{Per} evaluates any 1-periodic function in *turns*. The reduction mod 1 is exactly a digit decomposition, so it is verified at no extra cost, and one 2^{12} -row table with first-order interpolation reaches $\approx 1\Delta$. $\Pi_{\log}$ normalises multiplicatively with Msnzb , as Hao’s division does, and reaches 1.5Δ over $(0, 2^{60})$. Both come with error bounds and a security argument in the same hybrid model as [5]. We specialise them to snake, softplus and log-softmax.
3. **Implementation and evaluation** (Sections 5 and 6). We add the protocols to Hao et al.’s released C++ code (ZKMath on emp-zk [13]). A pure-Python executable specification comes with an adversarial prover, and the C++ output matches it bit for bit. On the real backend, every tampered witness is rejected. The benchmarks follow Hao et al.’s methodology: $10^3\text{--}10^5$ instances and 200 Mbps–1 Gbps links. We compare against the same functions proven *without* lookups in the same backend, and against plaintext.

What is (not) new. Evaluating log or sin through lookups is not new in itself. zkLLM’s *tlookup* handles arbitrary element-wise functions and uses a logarithm inside *zkAttn* [12]. Lookup-based non-linearities are standard in SNARK-based zkML [6]. Normalisation plus table lookup is the usual approach in secure computation [1, 10]. SwiGLU is directly expressible in Hao et al.’s framework and in zkLLM. Our contribution is to identify the pitfalls, to give sound and precise instantiations *inside* the digit-decomposition framework together with their proofs, and to provide an open, reproducible implementation and benchmark.

2 Background

Fixed point. We use Hao et al.’s parameters: $p = 2^{61} - 1$ and $s = 12$ fractional bits. A real $\hat{x}$ is encoded as $x = \text{R2F}(\hat{x}) = \lfloor \hat{x} \cdot 2^s \rfloor \bmod p$, and decoded as $\hat{x} = \text{F2R}(x) = x/2^s$, with x read as a signed integer in $(-p/2, p/2)$. Errors are stated in units of $\Delta = 2^{-s} \approx 2.44 \cdot 10^{-4}$. Public table entries are rounded to nearest (error $\leq \Delta/2$).

VOLE-based ZK. The prover P commits to values as IT-MACs $[x]$. Linear operations with public constants are free.

A multiplication $[x][y]$ costs one committed value and is checked in batches as in QuickSilver [16]. CheckZero opens a zero-test. We work in the $(\mathcal{F}_{\text{ZK}}, \mathcal{F}_{\text{ZK}}^{\text{Lookup}})$ -hybrid model of [5]. Lookup($L, [x], [\tilde{y}]$) checks that $(x, \tilde{y})$ is a row of a public table L , and Range($d, [x]$) checks that $x \in [0, 2^d]$. Both are realised with a ZK read-only memory [4]. N lookups into a T -row table cost $2N + T$ multiplications, i.e. about two per lookup, plus one committed version per read.

Building blocks [5]. DigitDec($[x]; d_0, \dots, d_{k-1}$) commits digits x_i , range-checks each of them (digits of up to 12 bits use one lookup), and runs CheckZero($\sum_i 2^{\sigma_i} x_i - x$) with $\sigma_i = \sum_{j < i} d_j$. Because $\sum d_i \leq 60 < \log p$, an accepted decomposition is unique and proves $x < 2^{\sum d_i}$. PosTrunc($[x], t$) is the high digit of DigitDec($[x]; t, n - t$), optionally after adding 2^{t-1} for rounding. We compute a signed truncation for $|x| < 2^L$ as PosTrunc($x + 2^L, t$) $- 2^{L-t}$, which costs one DigitDec. Msnzb proves $2^k \leq x < 2^{k+1}$; P commits k and the row $(k, 2^k, \dots)$ of a public table, and V range-checks $x - 2^k$ and $2^{k+1} - 1 - x$ on n bits. For $x < 2^n$ this is equivalent to Hao et al.’s comparisons. The exponential Π_{exp} computes $e^{-\hat{x}}$ from DigitDec($[x]; 12, 12, \dots$) with one table per digit, $L_i = \{v \mapsto e^{-2^{\sigma_i} v / 2^s}\}$, and multiplies the looked-up factors with truncation.

3 Two pitfalls

3.1 Digit-wise lookups need a homomorphism

Π_{exp} is correct because the digits of x split its value additively, $\hat{x} = \sum_i c_i$ with $c_i = 2^{\sigma_i} \hat{x}_i$, and the exponential maps sums to products. v1’s Π_{log} [15, Fig. 4.1] copies the structure. It looks up $y_i = \text{R2F}(\log c_i)$ in per-digit tables and outputs $\sum_i y_i$, justified by “the logarithm of a product is the sum of the logarithms”. The digits, however, combine as a sum.

Proposition 1. *With exact tables, v1’s Π_{log} outputs $\log \prod_i c_i$ instead of $\log \hat{x} = \log \sum_i c_i$, an error of $\log(\prod_i c_i / \sum_i c_i)$ whose size and sign depend on the input. The table entry for a zero digit would be $\log 0$ and does not exist, so every input with a zero digit makes the honest prover abort. Hence, for $k \geq 2$ digits, the protocol returns $\ln \hat{x}$ only by coincidence. With Hao et al.’s 12-bit digits ($k = 5$ for $x < 2^{60}$), every $x < 2^{48}$, i.e. every $\hat{x} < 2^{36}$, has a zero top digit and aborts.*

For example, $x = 2^{50} + 2^{40} + 3 \cdot 2^{24} + 5 \cdot 2^{12} + 7$ has five non-zero digits. v1 outputs 50.40 while $\ln \hat{x} = 26.34$. Every probability, every softmax denominator in $[1, n]$ and every softplus argument in $[1, 2]$ lies far below 2^{36} , so the honest prover cannot complete the protocol on the inputs it was designed for (last column of Table 1). The thesis text also writes “ $k = 12$ ”, which is the digit size d rather than the number of digits k .

3.2 Every witness must be pinned to the input

v1’s Π_{sin} [15, Fig. 4.2] works as follows. P computes $x_{\text{mod}} = \hat{x} - 2\pi \lfloor \hat{x} / 2\pi \rfloor$ locally and commits $[x_{\text{mod}}]$. It then commits $y = \text{R2F}(T_5(x_{\text{mod}}))$ with the Taylor polynomial $T_5(t) = t - t^3/6 + t^5/120$, and the parties run Lookup($L, [x_{\text{mod}}], [y]$).

Protocol $\Pi_{\text{Per}}[g]$ (1-periodic g , e.g. $\sin 2\pi\tau, \sin^2 2\pi\tau$)

Parameters: $b = 12$; table

$L_g = \{(i, \text{R2F}(g(i2^{-b})), \text{R2F}(g'(i2^{-b})))\}_{i < 2^b}$.

Input: $[u]$ with $|u| < 2^{58}$, read as $\tau = u/2^F$, $b \leq F \leq 48$.

1. $[u'] = [u] + 2^{58}$. Since $2^F \mid 2^{58}$, $\text{frac}(u'/2^F) = \text{frac}(\tau)$.
2. $([\delta], [i], [h]) \leftarrow \text{DigitDec}([u']; F - b, b, 59 - F)$. This is the verified reduction mod 1: $\text{frac}(\tau) = i2^{-b} + \delta 2^{-F}$; the top digit proves $|u| < 2^{58}$.
3. P commits $[g_0], [g_1]$; Lookup($L_g, [i], [g_0], [g_1]$).
4. $[y] = [g_0] + \text{Trunc}([g_1] \cdot [\delta], F)$ (first-order interpolation).

Figure 1: Evaluation of periodic functions in turns.

Protocol Π_{log}

Parameters: $n \leq 60, m = 12$; tables

$K = \{(k, 2^k, 2^{n-1-k}, \text{R2F}((k-s)\ln 2))\}_{k < n}$ and

$M = \{(z_1, \text{R2F}(\ln(z_1 2^{-m})), \text{R2F}(2^m/z_1))\}_{2^m \leq z_1 < 2^{m+1}}$.

Input: $[x]$ with $x \in (0, 2^n)$; output $\approx \ln \hat{x}$.

1. (Msnzb) P commits $[k], [e], [q], [c]$; Lookup($K, [k], [e], [q], [c]$); Range($n, [x] - [e]$); Range($n, 2[e] - 1 - [x]$).
2. $[z] = [x] \cdot [q]$, so $z \in [2^{n-1}, 2^n)$ (multiplicative normalisation).
3. $([z_0], [z_1]) \leftarrow \text{DigitDec}([z]; n - 1 - m, m + 1)$; P commits $[a], [b]$; Lookup($M, [z_1], [a], [b]$).
4. $[y] = [c] + [a] + \text{PosTrunc}_{\text{round}}([b] \cdot [z_0], n - 1)$.

Figure 2: Logarithm via normalisation (cf. Hao et al.’s Π_{Div}).

Proposition 2. *v1’s Π_{sin} is not sound. For every input $[x]$ and every row (t, y) of L , the prover that commits $x_{\text{mod}} = t$ is accepted with output y .*

Proof. $[x]$ does not appear in any check. The only check involving x_{mod} is the lookup, which passes for every row. $\square$

Our test suite runs this attack against a literal implementation of v1 and obtains an accepted proof of $\sin(0) = 1.0044$. Running the honest protocol also shows larger error margins than the thesis estimated. On the protocol’s domain $[0, 2\pi]$, T_5 is off by up to 46.5 at $t \rightarrow 2\pi^-$, and the degree-7 polynomial that the thesis suggests for a full cycle still errs by 30.2 there. Negative inputs wrap to $x_{\text{mod}} \approx 2\pi$, so $\hat{x} \in [-1, 0)$ already gives errors of 46. Moreover, a lookup-based protocol does not need a Taylor polynomial at all: the table is public, so it can store $\sin$ itself. The real difficulty is the *verified* range reduction. It cannot be exact modulo 2π , because 2π is irrational and a fixed-point constant drifts by $q \cdot |C2^{-s} - 2\pi|$ after q periods.

Lessons. (L1) Every committed value must be a function of the inputs, fixed by CheckZero, range and lookup relations. The soundness proofs in the hybrid model, including all of Hao et al.’s, rely on this *witness uniqueness*. (L2) A digit-wise lookup recombines correctly only if $f(\sum_i c_i) = \bigodot_i f(c_i)$ for a cheap operation $\bigodot$. This holds for exp with $\bigodot = \times$. It also holds for periodic functions whose period, in fixed-point units, is a power of two: the digits above the period contribute whole periods and drop out. For log one needs a multiplicative decomposition, i.e. normalisation.

Table 1: Accuracy in units of $\Delta = 2^{-12}$ over 10^5 random inputs (max / mean absolute error); cost per instance of Ours (function lookups + range checks / multiplications; log-softmax per element); and the v1 protocols [15] with an honest prover (max error or share of aborted runs).

Function	Ours	Naive	Cost	v1
$\sin x$	1.00/0.33	0.53/0.25	1+10/1	err 46
$\ln x, n=60$	1.49/0.40	1.03/0.33	2+22/2	err 49; abort 79%
$\ln x, n=20$	1.46/0.36	1.01/0.30	2+10/2	abort 100%
snake_1	1.00/0.33	0.54/0.25	1+14/1	err ≤ 2166
softplus	1.19/0.18	0.98/0.15	4+18/4	abort 100%
log-softmax ₁₀	3.82/0.54	2.99/0.46	2.2+8/2.1	abort 100%

4 Sound constructions

4.1 Periodic functions in turns

We parametrise periodic functions by *turns*. We write $\sin(a\hat{x}) = \sin(2\pi\tau)$ with $\tau = a\hat{x}/2\pi$, and fold the constant $a/2\pi$ into a public multiplier. For snake, a is a trained weight, so the constant can be folded in at training time. Concretely, $u = x \cdot A$ with $A = \text{round}(a2^{s_a}/2\pi)$ is a local operation and gives $\tau = u/2^{s+s_a}$. The reduction mod 1 is then the low F bits of u , which is exactly what DigitDec extracts. Figure 1 gives the protocol.

Lemma 1. *For $g \in C^2$ with period 1, Π_{Per} outputs y with $|\text{F2R}(y) - g(\tau)| \leq \Delta + 2^{-s-1-b} + \frac{1}{2} \max |g''| 2^{-2b}$. For $g = \sin 2\pi\tau$ and $b = 12$ this is 1.005Δ . The conversion from radians adds at most $2\pi|\hat{x}| |A2^{-s_a} - a/2\pi| \leq 2\pi|\hat{x}|2^{-s_a-1}$, which is $1.9 \cdot 10^{-7}$ for $|\hat{x}| < 4096$ and $s_a = 36$.*

Proof. Write $\tau_i = i2^{-b}$ and $\delta = \delta 2^{-F} \in [0, 2^{-b}]$. The table entries carry errors $|e_0|, |e_1| \leq \Delta/2$, and the rounded truncation adds $\Delta/2$. The term $e_1\delta$ is below 2^{-s-1-b} . The Taylor remainder of $g(\tau_i + \delta)$ around τ_i is at most $\max |g''| \delta^2/2$. $\square$

Cost: one lookup into a 2^{12} -row table, one multiplication, and the range lookups of two DigitDecs (59 and $F+5$ bits; 10 range lookups for $F = 48$, see Table 1). Snake, $\text{snake}_a(x) = x + \sin^2(ax)/a$, tabulates $g = \sin^2(2\pi\tau)$ directly, so no squaring is needed, and then multiplies by $1/a$ (a 24-bit constant) followed by one truncation. We treat a as public: it is fixed per channel after training. If a must stay private, $a/2\pi$ and $1/a$ are committed as model parameters, which costs one extra multiplication per activation.

4.2 Logarithm via normalisation

Figure 2 normalises exactly like the first steps of Hao et al.’s division. Msnzb gives k with $2^k \leq x < 2^{k+1}$, and multiplying by the looked-up 2^{n-1-k} yields $\hat{z} = z/2^{n-1} \in [1, 2)$ with $\ln \hat{x} = (k-s)\ln 2 + \ln \hat{z}$. The top $m+1$ bits z_1 of z index the mantissa table, and the remainder $r = z_0/2^{n-1} < 2^{-m}$ is handled by $\ln(\hat{z}_1 + r) = \ln \hat{z}_1 + r/\hat{z}_1 - \theta$, where $0 \leq \theta \leq 2^{-2m-1}$.

Lemma 2. *For every $x \in (0, 2^n)$, $|\text{F2R}(y) - \ln \hat{x}| \leq \frac{3}{2}\Delta + 2^{-2m-1} + 2^{-m-s-1}$, which is $3.66 \cdot 10^{-4}$ (1.5Δ) for $m = 12$.*

Proof. The entries c and a and the rounding of PosTrunc each contribute at most $\Delta/2$. The error e_b of b enters as $r e_b \leq 2^{-m-s-1}$, and the linearisation contributes θ . $\square$

Cost: 2 lookups (into n - and 2^{12} -row tables), 2 multiplications, and $2\lceil n/12 \rceil$ range checks for Msnzb plus those of two DigitDecs: 22 range checks for $n = 60$ and 10 for $n = 20$, two of them single-bit (Table 1). M contains no row with $z_1 < 2^m$, so the lookup itself enforces the normalisation. The protocol aborts on $x \leq 0$ and $x \geq 2^n$.

4.3 Activations

Softplus. We compute $\text{softplus}(x) = \text{ReLU}(x) + \ln(1 + e^{-|x|})$. A sign/absolute-value step costs one bit, one multiplication and one range check. Π_{exp} then evaluates $e^{-|x|}$, and Π_{log} runs with $n = s + 2$ because the argument lies in $[1, 2]$. **Log-softmax.** We compute $y_j = (x_j - m) - \ln \sum_k e^{x_k - m}$, where m comes from Hao et al.’s max (range checks plus $\prod_j (m - x_j) = 0$). The sum lies in $[1, n]$, so Π_{log} runs on $s + \lceil \log_2 n \rceil + 1$ bits, and no division is needed, unlike softmax. **Snake:** see above. Swish, GLU and SwiGLU need only Hao et al.’s sigmoid and a multiplication, and are not claimed here.

4.4 Security

We define the ideal functionality $\mathcal{F}_{\text{Math}}^+$ as Hao et al.’s $\mathcal{F}_{\text{Math}}$ extended with commands Log, Per_g , Snake, Softplus and LogSoftmax. Each command outputs the deterministic fixed-point function computed by the honest protocol, just as [5] defines PtExp, and aborts outside the domain. Lemmas 1 and 2 then bound the distance to the real functions.

Theorem 1. Π_{Per} , Π_{log} and the activation protocols UC-realise [3] $\mathcal{F}_{\text{Math}}^+$ in the $(\mathcal{F}_{\text{ZK}}, \mathcal{F}_{\text{ZK}}^{\text{Lookup}})$ -hybrid model against a static, malicious adversary.

Proof sketch. Corrupted verifier: V receives only accept/abort bits from the hybrid functionalities and never an opened value, so the simulator runs V on dummy inputs. *Corrupted prover:* the simulator extracts every committed witness from the Input calls and replays the checks. If all checks pass, the witnesses are the honest ones (L1). The digits of each DigitDec are unique because $\sum d_i \leq 60 < \log p$. Msnzb’s range checks fix k . Every table is a function of its key column. Products and linear combinations are determined. Hence the output equals the ideal output, and the simulator sends the extracted input to $\mathcal{F}_{\text{Math}}^+$. The structure follows the proofs in [5, App. B]. We cross-validate the argument empirically in Section 5. $\square$

5 Implementation

C++ on ZKMath. We extend Hao et al.’s released code (ZKMath, commit b4f03cd) on emp-zk. The emp-tool/-ot/-zk commits are pinned to December 2024, when ZKMath was published, and everything builds in a container. Apart from adding our files, the only change to the original code is the build type: ZKMath’s CMake file forces an unoptimised debug build, and we build everything with `-O3`. Our extension (939 lines of C++) adds a multi-column zk-ROM table. Unlike ZKMath’s tables, whose permutation-check challenges come from a fixed PRG key (acceptable for benchmarking, not for deployment), our tables obtain the challenges from the

Table 2: Runtime (s) and communication (MB) for 10^5 instances (log-softmax: 10^5 rows of 10), in the format of Hao et al.’s Tables 4–5. Speed-ups of Ours over the baseline in parentheses. Plain: floating-point evaluation without proof.

Function	Protocol	200 Mbps	500 Mbps	1 Gbps	Comm. (MB)	Elem./inst.
$\sin x$	Ours	1.610	0.663	0.350	37.2	49
	Poly	4.758 (3.0×)	2.144 (3.2×)	1.325 (3.8×)	108.2 (2.9×)	142
	Naive	9.777 (6.1×)	4.313 (6.5×)	2.627 (7.5×)	222.5 (6.0×)	292
	Plain		1.64 ms		–	–
$\ln x, n=60$	Ours	3.522	1.629	1.053	79.3	104
	Poly	6.190 (1.8×)	2.647 (1.6×)	1.515 (1.4×)	142.3 (1.8×)	187
	Naive	14.48 (4.1×)	6.461 (4.0×)	3.939 (3.7×)	326.8 (4.1×)	428
	Plain		0.84 ms		–	–
$\ln x, n=20$	Ours	1.671	0.760	0.467	38.5	50
	Poly	4.834 (2.9×)	2.092 (2.8×)	1.219 (2.6×)	110.4 (2.9×)	145
	Naive	10.13 (6.1×)	4.500 (5.9×)	2.679 (5.7×)	230.2 (6.0×)	302
	Plain		1.25 ms		–	–
snake_1	Ours	2.199	0.904	0.471	51.1	67
	Poly	5.984 (2.7×)	2.654 (2.9×)	1.577 (3.4×)	136.8 (2.7×)	179
	Naive	12.70 (5.8×)	5.631 (6.2×)	3.364 (7.2×)	288.4 (5.6×)	378
	Plain		2.85 ms		–	–
softplus	Ours	2.813	1.241	0.731	64.9	85
	Poly	9.598 (3.4×)	4.258 (3.4×)	2.547 (3.5×)	219.2 (3.4×)	287
	Naive	19.64 (7.0×)	8.670 (7.0×)	5.201 (7.1×)	446.1 (6.9×)	585
	Plain		2.78 ms		–	–
log-softmax ₁₀	Ours	15.51	6.910	4.159	351.2	46
	Poly	53.87 (3.5×)	23.88 (3.5×)	14.30 (3.4×)	1224 (3.5×)	160
	Naive	111.0 (7.2×)	49.19 (7.1×)	29.36 (7.1×)	2510 (7.1×)	329
	Plain		6.12 ms		–	–

verifier, at the cost of one extra round trip per 10^5 reads. The extension also adds a batched QuickSilver degree-2 check for bits, which commits no product, and the protocols of Section 4 together with the baselines of Section 6.

Executable specification. A pure-Python simulation of the protocols in the $(\mathcal{F}_{\text{ZK}}, \mathcal{F}_{\text{ZK}}^{\text{Lookup}})$ -hybrid model serves as the specification. It contains an adaptive malicious prover that shifts single witnesses, mounts carry attacks on adjacent digits, and tampers with random pairs of witnesses. Each accepted run must return the honest output. A mutation test confirms that this fuzzer does find accepted wrong outputs once any single check is disabled.

Cross-validation. (i) For every function, the C++ protocol run on the real VOLE backend outputs *exactly* the field elements of the Python specification (35,000 outputs, 0 mismatches). (ii) Malicious prover on the real backend: for each witness k of an execution, the prover adds 1 to witness k . Table 3 shows that all 878 such runs are rejected, including those against the baselines.

6 Evaluation

Setup. All runs use one AMD Ryzen 7 9800X3D 8-Core Processor machine with Linux 7.1, a Debian 12 container, GCC 12 and -O3. Following [5], both parties run single-threaded on one machine, and the loopback link is shaped to 200 Mbps, 500 Mbps and 1 Gbps. 10^5 instances is the default, and the parameters are $p = 2^{61} - 1$, $s = 12$ and 12-bit digits. Our times cover the whole proof *including*

Table 3: Malicious prover on the real emp-zk backend: each witness of one execution is incremented in turn. A sound protocol rejects the run or reproduces the honest output; “wrong” counts accepted runs with a different output.

Function	Protocol	Witnesses	Rejected	Wrong
$\sin x$	Ours	14	14	0
$\ln x, n=60$	Ours	31	31	0
$\ln x, n=20$	Ours	18	18	0
e^{-x}	Ours	8	8	0
snake_1	Ours	20	20	0
softplus	Ours	29	29	0
log-softmax ₁₀	Ours	119	119	0
$\sin x$	Poly	44	44	0
$\ln x, n=20$	Poly	45	45	0
$\sin x$	Naive	279	279	0
$\ln x, n=20$	Naive	271	271	0

every verifier check: lookup permutations, multiplication triples, CheckZero and degree-2 checks. Communication is the total over both directions.

Baselines. Ours uses lookups for range checks and for function evaluation. Poly keeps Hao et al.’s lookup-based building blocks, including Msnzb’s table indexed by the exponent, but replaces every *function* table by the smallest-degree polynomial with error below $\Delta/8$ (odd degree 7 for $\sin 2\pi t$ on $[0, \frac{1}{4}]$ after folding into the first quadrant, degree 5 for $\ln(1+t)$, and degree 4 for 2^{-f} with $e^{-x} = 2^{-k-f}$), evaluated by Horner’s rule at scale 2^{-20} . Naive additionally replaces every lookup-based range check by a bit decomposition with QuickSilver booleanity checks, and

Table 4: Calibration: Hao et al.’s released binaries (10^5 instances) re-run in our setup vs. their published numbers (500 Mbps).

Protocol	Comm. (MB)		Time (s), 500 Mbps	
	published	ours	published	ours
exp	99.0	28.7	8.70	0.49
div	110.7	110.7	9.84	2.09
sigmoid	189.9	189.9	17.71	3.64
gelu	338.2	432.2	32.70	8.34
softmax	816.3	816.2	78.29	15.95

Msnzb by a prefix-OR over the bits (one multiplication per bit). It is the “no table lookup” version of the same protocols: the same verified reductions in the same backend, with every lookup removed. The polynomial baselines are slightly *more* precise than ours (Table 1), so the comparison does not favour lookups. Plain is the floating-point function in C++.

Calibration. We re-ran Hao et al.’s own binaries (Table 4). The communication in both directions matches their published numbers within 0.1% for division, sigmoid and softmax, and for their building blocks: PosTrunc uses 15.57 MB and Msnzb 49.59 MB, against 15.571 and 49.586 MB in their Table 2. The released exponential takes 16-bit inputs, so it uses less communication than published. The released GELU uses 28% more, and we did not investigate the difference. Our absolute runtimes are 4–5 $\times$ lower than the published ones at the same bandwidth, because of a newer CPU and -O3. We therefore only compare protocols run in the same setup.

Results. Table 2 gives the results in the format of Hao et al.’s Tables 4–5. Lookups make the new math functions and activations 4.0–7.1 $\times$ faster than Naive and use 4.1–7.1 $\times$ less communication. Most of the gain already comes from lookup-based range checks (Poly vs. Naive). A 12-bit range check by lookup costs four committed values (the digit, its version and two permutation-check products), while a bit decomposition costs twelve. The function tables add a further 1.6–3.5 $\times$: they replace Horner steps whose truncations each need a 40-bit decomposition. At these bandwidths all protocols are communication-bound, so the speed-up closely follows the communication ratio. Only at 1 Gbps does computation start to matter, and the gap grows slightly (for sin, from 6.1 $\times$ at 200 Mbps to 7.5 $\times$ at 1 Gbps). With lookups, a proof costs 7–16 μ s per instance at 500 Mbps, compared with 6–29 ns for the plaintext function. The costs scale linearly with the number of instances, and the lookup advantage is already present at 10^3 instances, where the fixed cost of checking the 2^{12} -row tables is still visible (Table 5).

Why not 50–179 $\times$? Hao et al. compare against Mystique, which converts arithmetic values to Boolean ones and evaluates IEEE-754 circuits. We instead compare against the strongest simple arithmetic alternative in the *same* VOLE backend. The resulting factors of 4.0–7.1 $\times$ isolate what the lookup technique itself contributes to the new functions. They should not be read as a comparison with other systems.

Table 5: Scaling on the unshaped loopback: runtime (s), median of 3 runs.

Function	Protocol	10^3	10^4	10^5
sin x	Ours	0.006	0.032	0.210
	Naive	0.023	0.189	2.181
ln x , $n=60$	Ours	0.010	0.058	0.687
	Naive	0.031	0.270	3.398
ln x , $n=20$	Ours	0.007	0.038	0.319
	Naive	0.023	0.197	2.278
snake ₁	Ours	0.009	0.040	0.283
	Naive	0.028	0.234	2.846
softplus	Ours	0.009	0.059	0.472
	Naive	0.039	0.353	4.375
log-softmax ₁₀	Ours	0.032	0.212	2.803
	Naive	0.205	2.596	25.65

Limitations. Everything runs in the fixed-point model of [5] ($s = 12$), and our errors are quantisation-limited at $1-1.5\Delta$. Accuracy-critical uses need a larger s and larger tables. The proofs are for the protocols in the hybrid model. The C++ code is a research prototype built on an older emp-zk. We benchmark activation layers, not end-to-end networks.

7 Related work

Hao et al. [5] is the framework we build on. Mystique [14], Hao et al.’s baseline, provides arithmetic–Boolean conversions and evaluates floating-point functions with Boolean circuits. zkLLM [12] proves arbitrary element-wise functions with lookup on GPUs, and its attention protocol zkAttn uses a logarithm. Lu et al. [9] reduce non-linear layers to range and exponent relations. zkCNN covers ReLU and max-pooling [8]. SNARK-based zkML uses lookup tables [6], and Sridhar et al. give low-depth provable approximations of non-polynomial functions [11]. In MPC, SIRNN combines digit-wise lookups for exp with normalisation and lookups for reciprocal square roots [10], and Aly and Smart benchmark scientific functions such as sin and log built from range reduction and polynomials [1]. The VOLE-based ZK backends we use are surveyed in [2]. We add sound, precise instantiations of log and periodic functions *within* a digit-decomposition framework, together with a reusable account of the pitfalls of the direct instantiations.

8 Conclusion

Reusing a digit-wise lookup template for new functions is attractive, but some details are easy to miss on paper and only show up in an implementation. The template is sound only if every witness is pinned to the input, and correct only if the recombination is a homomorphism of the function. Following these two rules gives a logarithm and a periodic-function protocol at $1-1.5$ units in the last place, for $1-2$ lookups each, and makes softplus, log-softmax and snake available in Hao et al.’s framework at $4.0-7.1\times$ lower cost than without lookups. Code, specification, tests and benchmark scripts are released with this paper.

References

- [1] Abdelrahman Aly and Nigel P. Smart. Benchmarking privacy preserving scientific operations. In *Applied Cryptography and Network Security (ACNS)*, 2019.
- [2] Carsten Baum, Samuel Dittmer, Peter Scholl, and Xiao Wang. SoK: Vector OLE-based zero-knowledge protocols. Cryptology ePrint Archive, Paper 2023/857, 2023.
- [3] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 136–145, 2001.
- [4] Nicholas Franzese, Jonathan Katz, Steve Lu, Rafail Ostrovsky, Xiao Wang, and Chenkai Weng. Constant-overhead zero-knowledge for RAM programs. In *ACM Conference on Computer and Communications Security (CCS)*, pages 178–191, 2021.
- [5] Meng Hao, Hanxiao Chen, Hongwei Li, Chenkai Weng, Yuan Zhang, Haomiao Yang, and Tianwei Zhang. Scalable zero-knowledge proofs for non-linear functions in machine learning. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 3819–3836, 2024. Full version: Cryptology ePrint Archive 2025/507. Code: <https://github.com/CryptMatrix/ZKMath>.
- [6] Daniel Kang, Tatsunori Hashimoto, Ion Stoica, and Yi Sun. Scaling up trustless DNN inference with zero-knowledge proofs. arXiv:2210.08674, 2022.
- [7] Rithesh Kumar, Prem Seetharaman, Alejandro Luebs, Ishaan Kumar, and Kundan Kumar. High-fidelity audio compression with improved RVQGAN. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [8] Tianyi Liu, Xiang Xie, and Yupeng Zhang. zkCNN: Zero knowledge proofs for convolutional neural network predictions and accuracy. In *ACM Conference on Computer and Communications Security (CCS)*, 2021.
- [9] Tao Lu, Haoyu Wang, Wenjie Qu, Zonghui Wang, Jinye He, Tianyang Tao, Wenzhi Chen, and Jiaheng Zhang. An efficient and extensible zero-knowledge proof framework for neural networks. Cryptology ePrint Archive, Paper 2024/703, 2024.
- [10] Deevashwer Rathee, Mayank Rathee, Rahul Kranti Kiran Goli, Divya Gupta, Rahul Sharma, Nishanth Chandran, and Aseem Rastogi. SIRNN: A math library for secure RNN inference. In *IEEE Symposium on Security and Privacy (S&P)*, 2021.
- [11] Sriram Sridhar, Shravan Srinivasan, Dimitrios Papadopoulos, and Charalampos Papamanthou. Efficiently provable approximations for non-polynomial functions. Cryptology ePrint Archive, Paper 2025/2326, 2025.
- [12] Haochen Sun, Jason Li, and Hongyang Zhang. zkLLM: Zero knowledge proofs for large language models. In *ACM Conference on Computer and Communications Security (CCS)*, 2024.
- [13] Xiao Wang, Alex J. Malozemoff, and Jonathan Katz. EMP-toolkit: Efficient MultiParty computation toolkit. <https://github.com/emp-toolkit>, 2016.
- [14] Chenkai Weng, Kang Yang, Xiang Xie, Jonathan Katz, and Xiao Wang. Mystique: Efficient conversions for zero-knowledge proofs with applications to machine learning. In *30th USENIX Security Symposium (USENIX Security 21)*, 2021.
- [15] Zhi Chen Xiang. Design and extension of a zero-knowledge proof framework for non-linear functions in machine learning. Master’s thesis, Universidad Politécnica de Madrid, 2025.
- [16] Kang Yang, Pratik Sarkar, Chenkai Weng, and Xiao Wang. Quicksilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field. In *ACM Conference on Computer and Communications Security (CCS)*, 2021.
- [17] Liu Ziyin, Tilman Hartwig, and Masahito Ueda. Neural networks fail to learn periodic functions and how to fix it. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.